
title: "ISA591FirstStudyGuide"

output: html_document

date: "2026-05-09"

```
library(pROC)      # almost certain — ROC curves everywhere
library(randomForest) # almost certain — model comparison
library(skimr)      # very likely — data exploration
library(ggplot2)    # very likely — plotting
library(caret)      # likely — nearZeroVar()
library(Metrics)    # likely — rmse, mae, mape
library(corrplot)   # possible — multicollinearity check
library(cluster)    # possible — K-means work
library(gbm)
library(tree)
library(dplyr)      # filter, select, mutate — cleaner data wrangling
```

binary response = logistic regression

isnt a reponse variable (columns, no yes or no) = k means

Dimred: dimensionality, PCA, removing duplicate variables

Code Chunks

1. `include = FALSE` prevents code and results from appearing in the finished file. R Markdown still runs the code in the chunk, and the results can be used by other chunks.
2. `echo = FALSE` prevents code, but not the results from appearing in the finished file. This is a useful way to embed figures.
3. `eval = FALSE` prevents code from being executed. # {r, eval = FALSE}
4. `error = TRUE` permits errors in code.

Covariance matrix of variables

large variance = data is spread out a lot

small variance = data clustered together

```
``{r}
```

```
var(cereal$calories)
```

```
var(cereal$rating)
```

```
with(cereal, cov(calories, rating)) # positive covariance = as one increases, the other increases.
```

```
negative covariance = as one increases, the other decreases
```

```
# sum of variation of two variables
```

```
with(cereal, var(rating) + var(calories))
```

```
# if we dropped calories
```

```
with(cereal, var(calories)/(var(rating) + var(calories)))
```

```
# if we dropped rating
```

```
with(cereal, var(rating)/(var(rating) + var(calories)))
```

```
...
```

```
# analysis
```

```
``{r}
```

```
pcs <- prcomp(~ rating + calories, data = cereal)
```

```
summary(pcs)
```

```
# The summary above is telling us that the first principal component contains 86% of the original variation. The first principal component will always contain the most amount of variation.
```

```
# original rows, columns
```

```
dim(cereal)
```

```
# after PCA rows, columns
```

```
dim(pcs$x)
```

```
# calculate value of first PC for first observation
```

```
with(cereal, head(0.5315077 * (rating - mean(rating)) - 0.8470535 *(calories - mean(calories))))
```

```
# compare
```

```
head(pcs$x)
```

```
...
```

```
# Principle component analysis = reduces many variables into fewer variables
```

```
# Can answer : is the customer quiet or loud and squeaky wheels
```

```
# High outputs in PC1 (close to 1 or -1) show it is a loud customer and requires a lot of attention
```

```
# Squeaky wheels are shown when metrics have different outputs
```

```
``{r}
```

```
sasdat <- read.csv(file = "SAS_Data.csv")
```

```
pcs_sas <- prcomp(~., data=sasdat, center = TRUE, scale. = TRUE)
```

```
summary(pcs_sas) # output = variance explained/ importance of each component
```

```
pcs_sas$rotation # shows each variables contribution, values closer to 1 or -1 mean that the variable is driving the component
```

```
# output will show proportion of variance for the PC's, PC explains % variance and should keep increasing to next PC. should continue to add PC until cumulative hits 80-90%
```

```
...
```

```
# PCA example
```

```
``{r}
```

```
clean <- read.csv(file = "../Module3/vetdata_cleaned.csv")
```

```
clean_pr <- prcomp(~ GiftAvgLast + GiftAvg36 + GiftAvgAll + GiftAvgCard36,  
                  data = clean, center = TRUE, scale. = TRUE,  
                  na.action = na.omit)
```

```
summary(clean_pr)
```

```
# make a screen plot to decide on number of PCs to use
```

```
# looking for the elbow
```

Everything to the LEFT of where it flattens = keep those PCs. Everything after the elbow = those PCs add very little new information, not worth keeping.

```
plot(clean_pr)
```

```
...
```

Removing a variable

```
`{r}
```

```
df$variable_name <- NULL # removes the column
```

```
...
```

Reasons to remove

```
`{r}
```

Missing values

```
colSums(is.na(df))
```

```
summary(df)
```

No variation

```
library(caret)
```

```
nearZeroVar(df, saveMetrics = TRUE)
```

Multicollinearity = variables highly correlated with each other

```
library(corrplot)
```

```
cor_matrix <- cor(df, use = "complete.obs")
```

```
corrplot(cor_matrix, method = "color") # dark colors = strong relationships
```

Low importance score

```
library(randomForest)
```

```
rf_model <- randomForest(target ~ ., data = df, importance = TRUE)
```

```
varImpPlot(rf_model)
```

full overview of data

```
skimr()
```

```
...
```

Random functions

```
`{r}
```

Size of the dataset, rows and columns

```
dim(housing)
```

Summarize individual variables

```
summary(housing)
```

or

shows variable types (num, int, chr, factor)

```

str(housing)

# counts rows matching condition
sum(housing$YR.BUILT < 1700)

# removing a row that is bad data
dat <- with(housing, housing[YR.BUILT >= 1700, ])

# summarize individual variables graphically
numVars <- c("COUPON", "NEW", "HI", "S_INCOME", "E_INCOME", "S_POP", "E_POP",
"DISTANCE", "PAX", "FARE")
length(numVars)
catVars <- c("VACATION", "SW", "SLOT", "GATE")
length(catVars)
par(mfrow = c(2, 5))
for(var in numVars){
  hist(airfares[, var], xlab = var, main = "Histogram")
}
par(mfrow = c(2, 2))
for(var in catVars){
  barplot(table(airfares[, var]), main = var)
}
table(airfares[, "SW"])

# plot histogram for 6 variables
var_list <- c("HI", "S_INCOME", "E_INCOME", "S_POP", "E_POP", "DISTANCE")
par(mfrow= c(2, 3))
for(var in var_list){
  hist(airfares[, var], main = "Histogram")
}

...

``{r}
# plot to visually check
plot(housing$GROSS.AREA, housing$TOTAL.VALUE)

# straight line of best fit
mm <- lm(TOTAL.VALUE ~ GROSS.AREA, data = housing)
abline(a = coef(mm)[1], b = coef(mm)[2], col = "red", lwd = 2)

# fitting mode based on two variables
mm2 <- lm(TOTAL.VALUE ~ GROSS.AREA + LIVING.AREA, data = housing)
coef(mm2)

```

```
coef(mm)
```

```
...
```

```
# if there are a large amount of variables (>500), reduce the dimension
```

```
# Overfitting = when a model learns the training data too well, including its noise and quirks, and then performs poorly on new, unseen data.
```

```
# data partitioning
```

```
## 1:nrow is entire data set
```

```
## size is how much you want to sample using nrow * percentage 0.7=70%
```

```
## taking 70% of data set and use it to train rest of data
```

```
## data partitioning use replace = FALSE
```

```
## output of sample function is subset of integers or row index numbers
```

```
## minus train index is including whatever is not in the training index, so this is the validation set
```

```
```{r}
```

```
set.seed(13)
```

```
train.indx <- sample(1:nrow(dat), size = round(nrow(dat) * 0.7), replace = FALSE)
```

```
dat.train <- dat[train.indx,]
```

```
dat.val <- dat[-train.indx,]
```

```
head(dat.train)
```

```
...
```

```
build the model - linear regression model
```

```
```{r}
```

```
# Fit model with all variables
```

```
full <- lm(TOTAL.VALUE ~ ., data = dat.train)
```

```
# stepwise regression using BIC as the selection criterion
```

```
null <- lm(TOTAL.VALUE ~ 1, data = dat.train)
```

```
# letting r pick the best variables
```

```
# stepwise adds and removes variables to find the best combination
```

```
m.bic <- step(full, scope = list(lower = null, upper = full), direction = "both", trace = 0, k =
```

```
log(nrow(dat.train)))
```

```
...
```

```
# RMSE is how we compare models
```

```
```{r}
```

```
yhat.full <- predict(full, newdata = dat.val)
```

```
(rmse.full <- sqrt(mean((dat.val$TOTAL.VALUE - yhat.full)^2)))
```

```
yhat.m <- predict(m.bic, newdata = dat.val)
```

```
(rmse.m <- sqrt(mean((dat.val$TOTAL.VALUE - yhat.m)^2)))
```

```
...
```

```
MORE EXAMPLES
```

```

60/40 training and validation split
```{r}
set.seed(13)
trn.indx <- sample(1:nrow(dat), size = round(nrow(dat) * 0.6), replace = FALSE)
dat.trn2 <- dat[trn.indx, ]
dat.val2 <- dat[-trn.indx, ]

full2 <- lm(TOTAL.VALUE ~., data = dat.trn2)
null2 <- lm(TOTAL.VALUE ~ 1, data = dat.trn2)
m.bic2 <- step(full2, scope = list(lower = null2, upper = full2), direction = "both",
               trace = 0, k = log(nrow(dat.train2)))

yhat.full2 <- predict(full2, newdata=dat.val2)
rmse.full2 <- sqrt(mean((dat.val2$TOTAL.VALUE - yhat.full2)^2))
yhat.bic2 <- predict(m.bic2, newdata=dat.val2)
rmse.bic2 <- sqrt(mean((dat.val2$TOTAL.VALUE - yhat.bic2)^2))

c(rmse.full2, rmse.bic2)
```

RMSE analysis 100 times
```{r}
nrep <- 100

RMSEs.full <- rep(NA, nrep)
RMSEs.bic <- rep(NA, nrep)
set.seed(13)
for (irep in seq_len(nrep))
trn.indx <- sample(1:nrow(dat), size = round(nrow(dat) * 0.6), replace = FALSE)
dat.trn2 <- dat[trn.indx, ]
dat.val2 <- dat[-trn.indx, ]

full2 <- lm(TOTAL.VALUE ~., data = dat.trn2)
null2 <- lm(TOTAL.VALUE ~ 1, data = dat.trn2)
m.bic2 <- step(full2, scope = list(lower = null2, upper = full2), direction = "both",
               trace = 0, k = log(nrow(dat.train2)))

yhat.full2 <- predict(full2, newdata=dat.val2)
rmse.full2 <- sqrt(mean((dat.val2$TOTAL.VALUE - yhat.full2)^2))
yhat.bic2 <- predict(m.bic2, newdata=dat.val2)
rmse.bic2 <- sqrt(mean((dat.val2$TOTAL.VALUE - yhat.bic2)^2))

c(rmse.full2, rmse.bic2)
RMSEs.full[irep] <- rmse.full2

```

```
RMSEs.bic[irep] <- rmse.bic2
```

```
c(mean(RMSEs.full), sd(RMSEs.full)/sqrt(nrep))
```

```
c(mean(RMSEs.bic), sd(RMSEs.bic)/sqrt(nrep))
```

```
boxplot(RMSEs.full, RMSEs.bic, names = c("full", "BIC"), main = "Comparison of RMSEs")  
...
```

```
# Interpreting the results
```

Whichever has the lower RMSE wins on prediction performance

The BIC model is simpler — stepwise selection using BIC specifically penalizes complexity and removes variables that don't justify their inclusion, Fewer predictors = easier to interpret, explain, and maintain

```
# Deploy the model: actually using the model, making predictions
```

```
# Creating a regression model
```

```
```{r}
```

```
reg <- lm(FARE ~., data=airfares)
```

```
summary(reg)
```

```
reg$df.residual
```

```
predict(reg, newdata=airfares[1,])
```

```
airfares[1,]
```

```
predict(reg, newdata=tail(airfares, n=1))
```

```
tail(airfares, n=1)
```

```
...
```

```
Two types of Models
```

1. Predict a numerical value. The target variable is numerical.

2. Predict a class membership. The target variable is categorical.

```
to transform to a categorical variable instead of numerical : factor()
```

```
a good model is one that is close to the actual values
```

```
the difference between actual and predicted is called the residual
```

```
three measures
```

```
compute these measures on both the training and validation sets
```

```
positive = underestimated
```

```
negative = overestimated
```

```
lower RMSE is the better model
```

```
RMSE
```

```
```{r}
```

```

# Manual calculation
rmse <- sqrt(mean((dat.val$FARE - yhat)^2))
rmse
# Or using a package
library(Metrics)
rmse(actual = dat.val$FARE, predicted = yhat)
...

```

MAE

```

``` {r}
Manual calculation
mae <- mean(abs(dat.val$FARE - yhat))
mae
Or using a package
library(Metrics)
mae(actual = dat.val$FARE, predicted = yhat)
...

```

MAPE

```

``` {r}
# Manual calculation
mape <- mean(abs((dat.val$FARE - yhat) / dat.val$FARE))
mape * 100 # multiply by 100 to express as a percentage
# Or using a package
library(Metrics)
mape(actual = dat.val$FARE, predicted = yhat)
...

```

dealing with outliers

all three are effected by outliers

check by plotting residuals

```

``` {r}
residuals_val <- dat.val$FARE - yhat
Boxplot
boxplot(residuals_val, main = "Residuals on Validation Data")
Histogram
hist(residuals_val, main = "Residuals on Validation Data", xlab = "Residuals")
...

```

# or using median based measure

```

``` {r}
# Median Absolute Error - resistant to outliers
median(abs(dat.val$FARE - yhat))
...

```

```
# EXAMPLE WITH DATASET
```

```
`{r}
```

```
# split training and validation
```

```
set.seed(13)
```

```
train <- sample(1:nrow(toyota), size = round(nrow(toyota) * 0.6), replace = FALSE)
```

```
df.trn <- toyota[train, ]
```

```
df.val <- toyota[-train, ]
```

```
full <- lm(Price ~ . - Automatic - Quarterly_Tax, data = df.trn)
```

```
null <- lm(Price ~ 1, data = df.trn)
```

```
reg <- step(full, scope = list(lower = null, upper = full), direction = "both", trace = 0)
```

```
summary(reg)
```

```
# predicted values and RMSE
```

```
pred.trn <- predict(reg, newdata = df.trn)
```

```
rmse.trn <- with(df.trn, sqrt(mean((Price - pred.trn)^2)))
```

```
pred.val <- predict(reg, newdata = df.val)
```

```
rmse.val <- with(df.val, sqrt(mean((Price - pred.val)^2)))
```

```
print(c(rmse.trn, rmse.val))
```

```
# MAPE
```

```
pred.trn <- predict(reg, newdata = df.trn)
```

```
mape.trn = mean(abs(df.trn$Price - pred.trn) / df.trn$Price) * 100
```

```
pred.val <- predict(reg, newdata = df.val)
```

```
mape.val = mean(abs(df.val$Price - pred.val) / df.val$Price) * 100
```

```
mape.trn
```

```
mape.val
```

```
# box plots side by side
```

```
res.trn = df.trn$Price - pred.trn
```

```
res.val = df.val$Price - pred.val
```

```
boxplot(res.trn)
```

```
boxplot(res.val)
```

```
# Produce box-plots for both the training residuals and validation residuals, and put them side by side
```

```
trn_err <- df.trn$Price - pred.trn
```

```
val_err <- df.val$Price - pred.val
```

```
err <- c(trn_err, val_err)
```

```
err_type <- c(rep("train", length(trn_err)), rep("val", length(val_err)))
```

```
df_err <- data.frame(err_type = factor(err_type), err = err)
```

```
with(df_err, plot(err ~ err_type))
```

```
...
```

```
# calculating a confusion matrix
```

```
# confusion matrix: It's a table that shows how well your model classified things by comparing predicted vs actual labels
```

```
# TN (True Negative), Correctly predicted nonowner, Said nonowner, actually nonowner
```

```

# TP (True Positive), Correctly predicted owner,Said owner, actually owner
# FP (False Positive), Wrong — said owner, actually nonowner, Called them an owner but
they're not
# FN (False Negative), Wrong — said nonowner, actually owner, Missed an actual owner
```{r}
Let's calculate the confusion matrix using Prob.1 and a cutoff value of 0.5. Meaning only
those that receive a predicted probability of greater than 0.5 will be called an owner.
mower$cut1 <- ifelse(mower[, "Prob.1"] > 0.5, 'owner', 'nonowner')
(confmat <- with(mower, table(Ownership, cut1))) # building the confusion matrix
accuracy of the models
(accuracy <- sum(diag(confmat))/sum(confmat))
(misclass <- 1 - accuracy)
```

# classifications
# sensitivity
"Of all actual owners, how many did we correctly identify?"
```{r}
sens <- confmat[2,2] / sum(confmat[2,])
TP / (TP + FN)
```

# specificity
"Of all actual nonowners, how many did we correctly identify?"
```{r}
spec <- confmat[1,1] / sum(confmat[1,])
TN / (TN + FP)
```

# cutoff
Lower cutoff → catch more owners (higher sensitivity) but misclassify more nonowners (lower
specificity)
Higher cutoff → fewer false alarms (higher specificity) but miss more actual owners (lower
sensitivity)

# ROC Curves = plot of sensitivity
```{r}
library(pROC)
mower$Ownership <- relevel(mower$Ownership, ref = "nonowner")
r <- roc(mower$Ownership, mower$Prob.1) # builds the curve
names(r) # shows everything stored inside
print(r$auc) # AUC closer to 1 is better
library(ggplot2)
(p <- ggroc(r)) # plots the ROC - higher AUC is the better plot
```

```

If you are comparing models:

```
```{r}
r2 <- roc(mower$Ownership, mower$Prob.2)
r2$auc
roc <- list(r, r2)
(p <- ggroc(roc) + scale_colour_discrete("Model"))
```
```

To best split the two classes use `coords()`. This is the cutoff with the maximum sensitivity and specificity. Or in other words, the cutoff with the least amount of error.

```
```{r}
coords(r, "best") # finds the best cutoff
```
```

Which variable is categorical and which variable is numerical

numerical = num or int

categorical = chr or factor

```
```{R}
data <- read.csv('gpadat.csv')
str(data)
head(data)
library(skimr)
skim(data)

cat_names <- c()
num_names <- c()
for (varname in colnames(data)) {
 if(is.numeric(gpa[, varname])) {
 num_names <- append(num_names, varname)
 } else {
 cat_names <- append(cat_names, varname)
 }
}
print(cat_names)
print(num_names)
```
```

which variable contains missing value

```
```{R}
summary(data)
colnames(data)[colSums(is.na(data)) > 0]
simpler way
colSums(is.na(gpa))
```
```

```
# subset data so it only contains "B"
```

```
```{R}
```

```
grade_B <- subset(data, grade == "B") # simplest way to do it
```

```
grade_B <- data[which(data$grade == "B"),]
```

```
head(grade_B)
```

```
```
```

```
# Impute the missing values in `scores` by averaging the scores of the same grade.
```

```
```{R}
```

```
data$grades <- factor(data$grades, levels = c("A", "B", "C"))
```

```
mean_A <- mean(data$scores[data$grades == 'A'], na.rm=TRUE)
```

```
mean_B <- mean(data$scores[data$grades == 'B'], na.rm=TRUE)
```

```
mean_C <- mean(data$scores[data$grades == 'C'], na.rm=TRUE)
```

```
data$scores[is.na(data$scores) & data$grades == 'A'] <- mean_A
```

```
data$scores[is.na(data$scores) & data$grades == 'B'] <- mean_B
```

```
data$scores[is.na(data$scores) & data$grades == 'C'] <- mean_C
```

```
summary(data)
```

```
sum(is.na(data$scores))
```

```
grade2look <- "B"
```

```
mean(data[which(data$grades == grade2look), "scores"], na.rm = TRUE)
```

```
imputed <- data$scores
```

```
for(k in seq_len(nrow(data))) {
```

```
 if(is.na(data[k, "scores"])) {
```

```
 grade2look <- data[k, "grade"]
```

```
 imputed[k] <- mean(data[which(data$grades == grade2look), "scores"], na.rm = TRUE)
```

```
 }
```

```
}
```

```
cbind(data,imputed)
```

```
simplest way to impute
```

```
gpa$scores[is.na(gpa$scores) & gpa$grade == "A"] <- mean(gpa$scores[gpa$grade == "A"],
na.rm = TRUE)
```

```
gpa$scores[is.na(gpa$scores) & gpa$grade == "B"] <- mean(gpa$scores[gpa$grade == "B"],
na.rm = TRUE)
```

```
gpa$scores[is.na(gpa$scores) & gpa$grade == "C"] <- mean(gpa$scores[gpa$grade == "C"],
na.rm = TRUE)
```

```
```
```

```
# additional practice: using actual vs predicted, calculate MAE, MAPE, RMSE
```

```

```{r}
reading data file
sales <- read.csv(file = "sales.csv")
calculating the MAPE
(mape <- with(sales, mean(abs((ActualSales - PredictedSales)/ActualSales))) * 100)
use prediction errors boxplot to figure out if it is overestimating or underestimating
par(mfrow = c(1, 2))
with(sales, boxplot((ActualSales - PredictedSales)/abs(ActualSales)))
with(sales, hist((ActualSales - PredictedSales)/abs(ActualSales), xlab = "Predictor Error", main = "Histogram"))
plot shows There are more negative prediction errors than positive ones, so the model tends to overestimate sales
Calculate the median-based metrics: median absolute error, median absolute percentage error, and root median squared error
(medAE <- with(sales, median(abs(ActualSales - PredictedSales))))
(medAPE <- with(sales, median(abs((ActualSales - PredictedSales)/ActualSales))))
(RmedSE <- with(sales, sqrt(median((ActualSales - PredictedSales)^2))))

...

summarize a variable numerically
```{r}
airfares <- read.csv(file = "Airfares.csv", stringsAsFactors = TRUE)
head(airfares)
summary(airfares)

...

# summarize each individual variable graphically
```{r}
numVars <- c("COUPON", "NEW", "HI", "S_INCOME", "E_INCOME", "S_POP", "E_POP", "DISTANCE", "PAX", "FARE")
length(numVars) # amount of numerical variables
catVars <- c("VACATION", "SW", "SLOT", "GATE")
length(catVars) # amount of categorical variables

...

plot of numerical
```{r}
par(mfrow = c(2, 5))
for(var in numVars){
  hist(airfares[, var], xlab = var, main = "Histogram")
}
...

# plot of categorical
```{r}
par(mfrow = c(2, 2))

```

```

for(var in catVars){
 barplot(table(airfares[, var]), main = var)
}
...

```

# EXAMPLE

```

```{r}
# plot histogram of 6 variables
var_list <- c("HI", "S_INCOME", "E_INCOME", "S_POP", "E_POP", "DISTANCE")
par(mfrow = c(2, 3))
for (var in var_list) {
  hist(airfares[, var], xlab = var, main = "Histogram")
}

```

```

# regression model using y=FARE
reg <- lm(FARE ~ ., data = airfares)
summary(reg)

```

```

# calculate the residual
reg$df.residual

```

```

# using the regression model avg fare on route with same characteristics as first observation in
the dataset
predict(reg, newdata = airfares[1, ]) # predicting fare for route 1
predict(reg, newdata = head(airfares, n = 1)) # same thing just different way
head(airfares, n = 1) # shows the ACTUAL fare for the route

```

```

# do same thing but for the last observation
predict(reg, newdata = airfares[nrow(airfares), ])
# OR
predict(reg, newdata = tail(airfares, n = 1))
tail(airfares, n = 1)
...

```

```

# plotting two variables against each other
```{r}
plot(housing[, "GROSS.AREA"], housing[, "TOTAL.VALUE"])
OR
plot(housing$GROSS.AREA, housing$TOTAL.VALUE)

```

```

#OR
with(housing, plot(GROSS.AREA, TOTAL.VALUE)) # draws the scatterplot

```

```
mm <- lm(TOTAL.VALUE ~ GROSS.AREA, data = housing) # creates the regression line, a plot
only using gross area to predict total value
abline(a = coef(mm)[1], b = coef(mm)[2], col = "red", lwd = 2) # draws the regression line
```

```
mm2 <- lm(TOTAL.VALUE ~ GROSS.AREA + LIVING.AREA, data = housing) # uses gross area
and living area to predict total value
coef(mm2)
...`
```

```
EXAMPLE
```

```
`{r}
creating data frame with two variables
dat2 <- housing[, c("TOTAL.VALUE", "TAX")]
mean(dat2$TAX / (dat2$TOTAL.VALUE * 1000)) # average tax rate
...`
```

```
data partitioning
```

Training Partition: Used to build the model.

Validation Partition: Used to check the model for overfitting (sometimes used to select the best model).

Test Partition: Used to assess model performance.

```
data exploration and partition example
```

```
`{r}
read data
df <- read.csv("pva97nk.csv", stringsAsFactors = TRUE)
head(df)
correctly code all of the variables
Step 1: Correctly Code All the Variables
df$TARGET_B <- factor(df$TARGET_B, labels = c("No", "Yes")) # changes from 0/1 to no/yes
df$ID <- factor(df$ID) # converting to factor which is categorical
df$DemCluster <- factor(df$DemCluster) # demcluster treats as categories
look at summary stats
library(skimr)
skim(df)
replace 0's with NA's because a 0 means nothing to data and will mess up data
df[df$DemMedIncome == 0, "DemMedIncome"] <- NA
df[df$DemMedHomeValue == 0, "DemMedHomeValue"] <- NA
df[which(df$DemAge == 0), "DemAge"] <- NA # proposed by Section A
basic plots
sort values into two lists, seperate num and cat
num_names <- c()
cat_names <- c()
for(varname in colnames(df)) {
```

```

if(is.factor(df[, varname])) {
 cat_names <- append(cat_names, varname)
}
if(is.numeric(df[, varname])) {
 num_names <- append(num_names, varname)
}
}
length(num_names)
length(cat_names)
for loop to get histogram of numerical variables
par(mfrow = c(6, 4))
for (varname in num_names) {
 hist(df[, varname], xlab = varname)
}
for loop to get bar plot of categorical variable except ID
cat_names <- setdiff(cat_names, "ID")
par(mfrow = c(2, 3))
for(varname in cat_names) {
 barplot(table(df[, varname]), main = varname)
}
pairwise relationships
plotting each numerical against the response variable TARGET_B
bigger the difference in the boxplots the more predictive power
par(mfrow = c(6, 4))
for (varname in num_names) {
 plot(df[, "TARGET_B"], df[, varname], ylab = varname, xlab = "TARGET_B")
}
take the logarithim of variables and produce boxplot s
df$LogGiftAvgLast <- log(df$GiftAvgLast + 1)
df$LogGiftAvg36 <- log(df$GiftAvg36 + 1)
df$LogGiftAvgAll <- log(df$GiftAvgAll + 1)
df$LogGiftAvgCard36 <- log(df$GiftAvgCard36 + 1)
par(mfrow = c(2, 2))
plot(df[, "TARGET_B"], df$LogGiftAvgLast, ylab="LogGiftAvgLast", xlab="TARGET_B")
plot(df[, "TARGET_B"], df$LogGiftAvg36, ylab="LogGiftAvg36", xlab="TARGET_B")
plot(df[, "TARGET_B"], df$LogGiftAvgAll, ylab="LogGiftAvgAll", xlab="TARGET_B")
plot(df[, "TARGET_B"], df$LogGiftAvgCard36, ylab="LogGiftAvgCard36", xlab="TARGET_B")
OR loop that does the same thing
names2trsf <- c("GiftAvgLast", "GiftAvg36", "GiftAvgAll", "GiftAvgCard36")
for (varname in names2trsf) {
 temp_var <- log(1 + df[, varname])
 df <- cbind(df, temp_var)
 colnames(df)[ncol(df)] <- paste("Log", varname, sep = "")
}

```

```

par(mfrow = c(2, 2))
for (varname in names2trsf) {
 logvarname <- paste("Log", varname, sep = "")
 plot(df[, "TARGET_B"], df[, logvarname], xlab = "TARGET_B", ylab = logvarname)
}
head(df)
relationship between categorical variables
output shows no and yes columns, if they are super similar in amounts then there is not a
relationship, if they differ then there is a relationship
for (varname in cat_names) {
 if((varname != "TARGET_B") && (varname != "DemCluster")){
 print(varname)
 print(table(df[, varname], df[, "TARGET_B"]))
 }
}
removing missing values
casesWithNA <- rowSums(is.na(df)) > 0
sum(casesWithNA) # total number of cases with NA
df.rm <- df[!casesWithNA,]
nrow(df.rm)/nrow(df) # many observations would be dropped
impute values based on mean and median for the missing values
median imputation
df$Impute_DemMedIncome <- df$DemMedIncome
df$Impute_DemMedIncome[is.na(df$DemMedIncome)] <- median(df$DemMedIncome,
na.rm=TRUE)
summary(df$DemMedIncome)
summary(df$Impute_DemMedIncome)
mean imputation
df$Impute_DemAge <- df$DemAge
df$Impute_DemAge[is.na(df$DemAge)] <- mean(df$DemAge, na.rm=TRUE)
summary(df$DemAge)
summary(df$Impute_DemAge)
automate imputation across columns
loggednames <- paste("Log", names2trsf, sep = "")
(impute_names <- c(num_names, loggednames))
for (varname in impute_names) {
 if (any(is.na(df[, varname]))) {
 imputed <- df[, varname]
 imputed[is.na(df[, varname])] <- mean(df[, varname], na.rm = TRUE)
 df <- cbind(df, imputed)
 colnames(df)[ncol(df)] <- paste("Imputed", varname, sep = "_")
 }
}
head(df)

```

```

write a for loop to replace missing values for categorical variables
(toy_df <- data.frame(x = c(1, NA, 3), y = c("A", "B", NA), z = c(NA, "F", "M")))
for (k in seq_len(ncol(toy_df))) {
 if (!is.numeric(toy_df[, k])) {
 toy_df[is.na(toy_df[, k]), k] <- "Unknown"
 }
}
toy_df

```

```

creating a factor
quarterly <- factor(c("Q1", "Q2", "Q3", "Q4"))
quarterly[1:4] <- "Q1"
print(quarterly)
[1] Q1 Q1 Q1 Q1
Levels: Q1 Q2 Q3 Q4
checking if its a factor
is.factor(quarterly)
[1] TRUE
combining two factors
c(quarterly, factor(c("Q2", "Q2")))
[1] Q1 Q1 Q1 Q1 Q2 Q2
Levels: Q1 Q2 Q3 Q4
creating character instead
c(quarterly, factor(c("Q2", "Q2")))
[1] Q1 Q1 Q1 Q1 Q2 Q2
Levels: Q1 Q2 Q3 Q4
if you assign Q5 to first value in quarterly then you get an error

```

```

log transformed of skewed variables
df$GiftAvg36 <- log(df$GiftAvg36 + 0.1)
df$GiftAvgLast <- log(df$GiftAvgLast + 0.1)
df$GiftAvgAll <- log(df$GiftAvgAll)
df$GiftAvgCard36 <- log(df$GiftAvgCard36)
check distributions with histograms
skewnames <- c("GiftAvg36", "GiftAvgLast", "GiftAvgAll", "GiftAvgCard36")
par(mfrow = c(2, 2))
for (varname in skewnames) {
 hist(df[, varname], xlab = varname, main = "Histogram")
}

```

```

merging clusters into groups
cbnCluster <- pva$DemCluster
cbnCluster[(pva$DemCluster >= 0) & (pva$DemCluster <= 17)] <- 1

```

```

cbnCluster[(pva$DemCluster >= 18) & (pva$DemCluster <= 35)] <- 2
cbnCluster[(pva$DemCluster >= 36) & (pva$DemCluster <= 53)] <- 3
cbnCluster <- factor(cbnCluster)
...

```

# curse of dimensionality

Curse of Dimensionality — too many variables and not enough data rows makes your model unreliable.

Increase n — get more data rows to support the number of variables you have.

Decrease p — remove variables until the number of predictors is manageable for your dataset size.

Timing — don't use a variable in your model if you won't have access to it when making real predictions.

Cost — if a variable is expensive to collect, replace it with a cheaper correlated variable instead.

Almost no variation — remove columns where nearly every row has the same value because they tell the model nothing.

Too many missing values — remove columns that are mostly NA because there isn't enough real data to make them useful.

# reducing categorical variables

```

```{r}
# read data
pva <- read.csv(file = "pva97nk.csv", stringsAsFactors = TRUE)
CbnCluster <- factor(pva[, "DemCluster"])
# counts how many people are in each cluster
sort(table(CbnCluster), decreasing = TRUE)
# makes a list of clusters that are too small to be on their own
(clusters2cbn <- names(table(CbnCluster))[table(CbnCluster) < 300] )
levels(CbnCluster)[levels(CbnCluster) %in% clusters2cbn] <- "Other" # collapse all the low-freq
df <- cbind(pva, CbnCluster) # add the combined cluster variable to the data frame.
head(df[, c("DemCluster", "CbnCluster")], n = 20)
...

```

With the toydat given below, create a variable that collapse the two lowest-frequency categories of variable y

```

```{r}
set.seed(22)
toydat <- data.frame(x = 1:10, y = factor(sample(LETTERS[1:3], size = 10, replace = TRUE)))
print(toydat)
newy <- toydat$y
(names2cbn <- names(sort(table(toydat$y), decreasing = FALSE))[1:2])
levels(newy)[levels(newy) %in% names2cbn] <- "Other"
head(cbind(toydat, newy))
...

```

# reading a confusion matrix

##            predicted

## actual   nonowner   owner

## nonowner   10       2     ← TN=10, FP=2

## owner       2       10    ← FN=2, TP=10

Diagonal (top-left, bottom-right) = correct predictions

Off-diagonal = mistakes

accuracy =  $(10+10)/24 = 83.3\%$

sensitivity =  $10/(2+10) = 83.3\%$  of real owners caught

specificity =  $10/(10+2) = 83.3\%$  of nonowners correctly identified